

Ejercicio 1 – Recursividad

Dada la siguiente definición para n y m , enteros no negativos:

$$T(n, m) = \begin{cases} n + m^2, & n < 4 \wedge m < 2 \\ n^2 + m, & n < 4 \vee m < 2 \\ 3 * T(n - 1, m - 1) + 2, & n \text{ par} \wedge n \geq 4 \wedge m \geq 2 \\ T(n - 1, m - 2) + T(n - 2, m - 2), & \text{eoc} \end{cases}$$

Resuelva los siguientes apartados:

- Implemente en Java la solución recursiva sin memoria y con memoria, para $T(n, m)$
- Transforme la solución implementada en el apartado a) a una solución iterativa.

Tiempo estimado: 40 min.

Puntuación: ADDA: 25%. EDA: 33,3%.

SOLUCIÓN

Recursivo sin memoria:

```
public static Integer T(Integer n, Integer m) {
    Integer res;
    if (n < 4 && m < 2) {
        res = n + (m * m);
    } else if (n < 4 || m < 2) {
        res = (n * n) + m;
    } else if (n % 2 == 0 && n >= 4 && m >= 2) {
        res = 3 * T(n - 1, m - 1) + 2;
    } else {
        res = T(n - 1, m - 2) + T(n - 2, m - 2);
    }
    return res;
}
```

Recursivo con memoria:

```
public static Integer T_memoria(Integer n, Integer m) {
    return T_memoria_aux(n, m, new HashMap<>());
}
```

```
public static Integer T_memoria_aux(Integer n, Integer m,
Map<Pair<Integer, Integer>, Integer> ac) {
    Integer res = ac.get(Pair.of(n, m));
    if (res == null) {
        if (n < 4 && m < 2) {
            res = n + (m * m);
        } else if (n < 4 || m < 2) {
            res = (n * n) + m;
        } else if (n % 2 == 0 && n >= 4 && m >= 2) {
            res = 3 * T_memoria_aux(n - 1, m - 1, ac) + 2;
        } else {
            res = T_memoria_aux(n - 1, m - 2, ac) +
T_memoria_aux(n - 2, m - 2, ac);
        }
        ac.put(Pair.of(n, m), res);
    }
    return res;
}
```

Iterativo:

```
public static Integer T_iterativo(Integer n, Integer m) {
    Map<Pair<Integer, Integer>, Integer> ac = new HashMap<>();
    Integer res = null;
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= m; j++) {
            if (i < 4 && j < 2) {
                res = i + (j * j);
            } else if (i < 4 || j < 2) {
                res = (i * i) + j;
            } else if (i % 2 == 0 && i >= 2 && j >= 2) {
                res = 3 * ac.get(Pair.of(i - 1, j - 1)) + 2;
            } else {
                res = ac.get(Pair.of(i - 1, j - 2)) +
ac.get(Pair.of(i - 2, j - 2));
            }
            ac.put(Pair.of(i, j), res);
        }
    }
    return ac.get(Pair.of(n, m));
}
```